

Real-Time Group Chat Application

System Architecture, Security Design & Lab Deployment Report

Course Assignment: Real-Time Group Chat	Target Environment: Laboratory SSH Server
Host Machine: student@10.1.75.51 (Port 2237)	Internal Server Port: 5000
External Public URL: http://10.1.75.51:5237/	Tech Stack: Flask, Flask-SocketIO, SQLite, AES-256-GCM, Ed25519

1. Executive Summary & Lab Network Mapping

This project implements a multi-user, real-time Group Chat Application using WebSockets (Flask-SocketIO) and SQLite persistence with end-to-end security enhancements (AES-256-GCM confidentiality and Ed25519 digital signatures). The application backend runs on the assigned SSH laboratory host (student@10.1.75.51).

Important Port Mapping Note for Evaluation:

While the Flask backend process binds internally to port 5000, the laboratory NAT router maps SSH port 2237 to external web port 5237. Therefore, the official testing URL accessible to all lab clients and TAs is: <http://10.1.75.51:5237/>.

2. Group Members & SSH Machine Allocation

Role	Student Member	SSH Server Connection	Mapped Public Web Port
Group Head / Host	Member 1 (Host Server)	ssh -p 2237 student@10.1.75.51	http://10.1.75.51:5237/
Client Member 2	Member 2	ssh -p 2238 student@10.1.75.51	Accesses http://10.1.75.51:5237/
Client Member 3	Member 3	ssh -p 2239 student@10.1.75.51	Accesses http://10.1.75.51:5237/
Client Member 4	Member 4	ssh -p 2240 student@10.1.75.51	Accesses http://10.1.75.51:5237/

3. Feature Coverage & Verification Matrix

Requirement / Feature	Implementation Details	Verification Result
Real-Time Messaging	Bi-directional WebSocket event broadcasting via Flask-SocketIO	PASSED
Join/Leave Notifications	Automatic broadcast of user arrival/departure pills to room	PASSED
User Identification	Unique username enforcement & dynamic avatar generation	PASSED
Graceful Disconnections	Server socket cleanup & broadcast on browser tab close	PASSED
SQLite Message Persistence	Stored in chat.db; auto-loaded upon room entry/reconnect	PASSED
Confidentiality (AES-256-GCM)	Payloads encrypted with room key before database insertion	PASSED
Authenticity (Ed25519)	Cryptographic digital signature per user stored & verified	PASSED
Public Access URL	External port forwarding mapped to http://10.1.75.51:5237/	PASSED

4. System Architecture Overview

Architectural Workflow:

1. **Client Frontend:** Browser connects via Socket.IO client over `http://10.1.75.51:5237/`.
2. **Port Forwarding:** External port 5237 routes incoming traffic to internal Flask port 5000.
3. **WebSocket Handler (Flask-SocketIO):** Manages connection lifecycle (join, leave, message, typing).
4. **Security Subsystem:** Encrypts message body using AES-256-GCM and signs plaintext using sender's Ed25519 key.
5. **Persistence Layer (SQLite):** Writes record to `chat.db` (tables: `messages`, `signing_keys`).

5. Database Schema & SQLite Inspection Guide

Messages are persisted in `chat.db` under the following schema:

```
CREATE TABLE messages (id INTEGER PRIMARY KEY, room_id TEXT, sender TEXT, ciphertext TEXT, nonce TEXT, signature TEXT, timestamp TEXT);
```

To inspect stored messages and signatures via SSH:

```
sqlite3 ~/chat_app/chat.db "SELECT id, sender, ciphertext, signature, timestamp FROM messages;"
```

6. Submission Links & Verification Summary

- **GitHub Source Code Repository:** <https://github.com/chaitanyakumarAI/Group-Chat>
- **Live Application URL (For TA Testing):** <http://10.1.75.51:5237/>
- **SSH Server Host Command:** `ssh -p 2237 student@10.1.75.51`
- **Background Daemon Command:** `nohup python3 server.py > server.log 2>&1 &`